

Predictive Sales Analytics Engine: From Classical ML Baselines to a Confidence-Gated Neuro-Symbolic Hybrid for B2B SaaS Conversations

Vivekananda
Enrollment No. 230077

Manasa
Enrollment No. 230078

Abstract—We try to predict whether a B2B SaaS sales deal is Won or Lost. Each deal has two kinds of input: 75 numeric CRM features (deal size, engagement, etc.) and a full sales conversation between the rep and the customer. The dataset has 8,000 deals from DeepMostInnovations/saas-sales-conversations.

We did this in three phases. Phase 1 used standard models. A Random Forest on the 75 features got $F1 = 0.9638$ and a simple average of three models got $F1 = 0.9710$. But a text model that compressed the conversation to 20 numbers (TF-IDF $\rightarrow$ PCA-20) only got $F1 = 0.5547$. So the text branch was broken because of too much compression.

Phase 2 fixed the text branch. We replaced the PCA-20 step with a frozen DistilBERT model that gives a 768-dim vector per conversation. Text-only $F1$ jumped to 0.7048. We also built a new fusion block called Gated Cross-Modal Attention (GCMA). In this block the tabular features ask questions to the text and a learned gate decides how much to trust the text per sample. A simple Concat fusion got $F1 = 0.9749$. We ran a careful ablation (turning each part on and off one at a time) and added three training tricks: MixUp, curriculum learning, and self-supervised pretraining. Each trick gave a small extra gain.

Phase 3 put the Random Forest and the neural model together using a small confidence gate. The gate is a tiny neural network that looks at three numbers: the RF prediction, how much the RF trees disagree (this tells us when the RF is unsure), and the neural prediction. It then outputs a weight w between 0 and 1 and mixes the two predictions. The hybrid got $F1 = 0.9749$, fixed 17 mistakes, and made zero new ones compared to the RF. On the half of the test set where the RF trees disagreed most, the hybrid lifted $F1$ from 0.9518 (RF) to 0.9606.

The whole project is fully reproducible with a setup.sh script, a Dockerfile, and a GitHub Actions CI pipeline.

Index Terms—Sales forecasting, B2B SaaS, multi-modal fusion, cross-modal attention, gated fusion, neuro-symbolic AI, epistemic uncertainty, stacking, confidence gating, DistilBERT, Random Forest, ablation study, interpretability.

I. Introduction

A. Problem Statement

Each sales deal has two kinds of input: 75 numeric CRM features (deal size, engagement, stage timings, product and channel categories) and a full sales conversation. The goal is to predict Won or Lost. We use 8,000 deals from DeepMostInnovations/saas-sales-conversations [1], sampled with random_state=42 so all phases use the same data.

B. Three Phases at a Glance

Figure 1 shows how the project grew across three phases. Each phase fixes a problem found in the one before it.

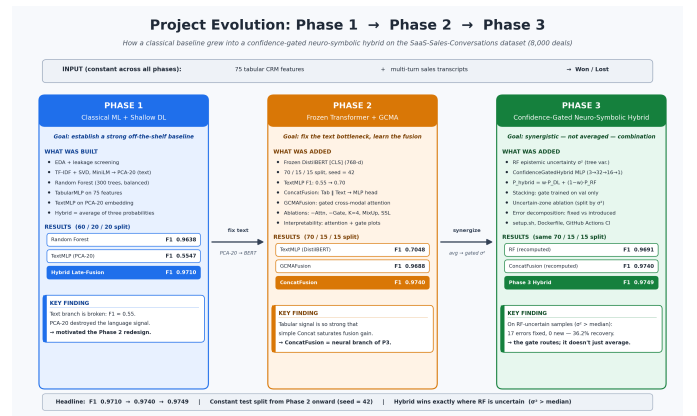


Fig. 1. Project evolution across three phases. Phase 1 builds a strong baseline but the text branch is broken. Phase 2 fixes the text branch and adds gated cross-modal attention. Phase 3 combines the symbolic and neural branches using a learned confidence gate.

C. Contributions

- **Step-by-step diagnosis.** Phase 1 shows that PCA-20 is the reason the text branch is so weak, which leads directly to Phase 2.
- **Gated Cross-Modal Attention (GCMA).** A new fusion block where the tabular branch asks questions to K learned views of the conversation, and a sigmoid gate decides how much of the text answer to use.
- **A clean ablation study.** Seven Phase 2 variants (Full GCMA, $-$ Attention, $-$ Gate, $K=4$, $+MixUp$, $+Curriculum$, $+SSL$ pretrain) on the same train/val/test split.
- **Confidence-Gated Neuro-Symbolic Hybrid (Phase 3).** A small gate that uses Random-Forest tree disagreement as an uncertainty signal and learns when to trust the symbolic branch versus the neural branch.
- **Two diagnostic tests for synergy.** (a) Split the test set by RF tree variance and check where the gain comes from; (b) count how many errors the hybrid fixes versus how many it introduces.

- Easy reproducibility. setup.sh for local installs, a Dockerfile for containers, and a GitHub Actions CI run on every push.

II. Dataset and Feature Pipeline

A. Dataset

We use 8,000 deals from DeepMostInnovations/saas-sales-conversations [1] with random_state=42. The classes are roughly balanced (49.75% Won, 50.25% Lost). Phase 1 used a 60/20/20 train/val/test split. Phases 2 and 3 share a 70/15/15 stratified split (5,600 train / 1,200 val / 1,200 test) so the numbers can be compared directly.

B. Tabular Features (same in every phase)

We one-hot encode the categorical columns (product, channel, conversation style and flow), take the log of the skewed count features (text length, word count, conversation length), standardise the continuous features (engagement, effectiveness), and add a few engagement×length interaction features. The result is 75 columns per conversation.

C. Post-Split Rescaling (a Phase 2 fix)

Phase 1 left three raw count columns (text_length, word_count, conversation_length) unscaled next to their log-standardised versions. This caused unstable gradients in single-modality and concat models. So in Phase 2 we add an extra StandardScaler fit on training data and applied to all splits. After this fix, X_{tab} has mean 0, std 1, and a max value around 5.3 (down from 2563).

D. Text Representations

- Phase 1: TF-IDF $\rightarrow$ Truncated SVD, then compressed with PCA down to 20 numbers per conversation.
- Phase 2 / Phase 3: we tokenise each conversation with the distilbert-base-uncased tokenizer at max_length= 512, then run it through DistilBERT in eval mode (no gradients, no fine-tuning). We take the [CLS] token from the last hidden state as a 768-dim vector per conversation. The encoder is just a fixed feature extractor. We cache the result to data/processed/saas_features_pretrained.parquet so we only run DistilBERT once.

III. Phase 1 – Classical ML and Shallow DL Baseline

A. Models

- Random Forest (RF): 300 trees, balanced class weights, max_features=sqrt, random_state=42, on the 75 tabular features.
- TabularMLP: 75 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 1 with ReLU and Dropout(0.3), same 75 features.
- TextMLP (PCA-20): an MLP on the 20-d compressed text vector.
- Hybrid Late Fusion: simple average of the three model probabilities.

TABLE I
Phase 1 – baselines.

Model	Test F1
Random Forest (tabular)	0.9638
TabularMLP (75 features)	0.9655
TextMLP (PCA-20)	0.5547
Hybrid Late-Fusion (mean of 3)	0.9710

B. Phase 1 Results (60/20/20 split)

C. What Went Wrong – the PCA-20 Bottleneck

The text branch was basically broken: $F1 \approx 0.55$ is barely above just guessing (since the classes are 50/50). We tried bigger 3,072-dim MiniLM embeddings through the same PCA-20 step. Text-only F1 only moved to 0.6215. So the problem was not the embedding – the problem was squeezing everything down to just 20 numbers. This single finding is what motivated all of Phase 2.

IV. Phase 2 – Gated Cross-Modal Attention (GCMA)

A. Why a New Fusion Block

Phase 1’s hybrid just averaged three model predictions. Phase 2 instead lets the tabular branch ask questions of the conversation, and a learned gate decides how much of the answer to trust per sample. Idea: a short high-engagement deal should weight the conversation differently from a long enterprise pitch.

B. Learned Text Views

A single [CLS] vector is too small to attend over (softmax over one item always returns 1). So we first build K learned views of the text. Let $h_{\text{text}} \in \mathbb{R}^{768}$ be the [CLS] vector and $W_{\text{views}} \in \mathbb{R}^{(K \cdot d_h) \times 768}$ a single linear layer:

$$H_{\text{views}} = \text{reshape}(W_{\text{views}} h_{\text{text}}, (K, d_h)) \in \mathbb{R}^{K \times d_h}. \quad (1)$$

We use $K = 8$ views of size $d_h = 128$.

C. Cross-Attention

Let $h_{\text{tab}} \in \mathbb{R}^{d_h}$ be the output of a 2-layer MLP on the tabular features. We use it as the query and use H_{views} as the keys and values:

$$Q = W_Q h_{\text{tab}}, \quad K_a = W_K H_{\text{views}}, \quad V = W_V H_{\text{views}}, \quad (2)$$

$$\alpha = \text{softmax}\left(\frac{QK_a^\top}{c\sqrt{d_h}}\right) \in \Delta^{K-1}, \quad (3)$$

$$z_{\text{text}} = \alpha^\top V \in \mathbb{R}^{d_h}. \quad (4)$$

We use a temperature $c = 2$. With the standard $\sqrt{d_h}$ scaling, the attention almost immediately picked one view (entropy ≈ 0.003 nats) and the gradients died. Doubling the denominator keeps the attention soft enough to train.

D. Sigmoid Gate and Fusion

The gate g is computed from the concatenation of h_{tab} and z_{text} :

$$g = \sigma(W_g [h_{\text{tab}}; z_{\text{text}}]) \in [0, 1]^{d_h}. \quad (5)$$

W_g is initialised with Xavier-normal (gain 0.1, zero bias), so $g \approx 0.5$ at the start. The fused vector is

$$z = \text{LayerNorm}(g \odot z_{\text{text}} + (1 - g) \odot W_p h_{\text{tab}}), \quad (6)$$

where W_p projects h_{tab} into $\mathbb{R}^{d_h}$. A small classifier head $d_h \rightarrow 64 \rightarrow 1$ gives the final logit, which goes into BCEWithLogitsLoss.

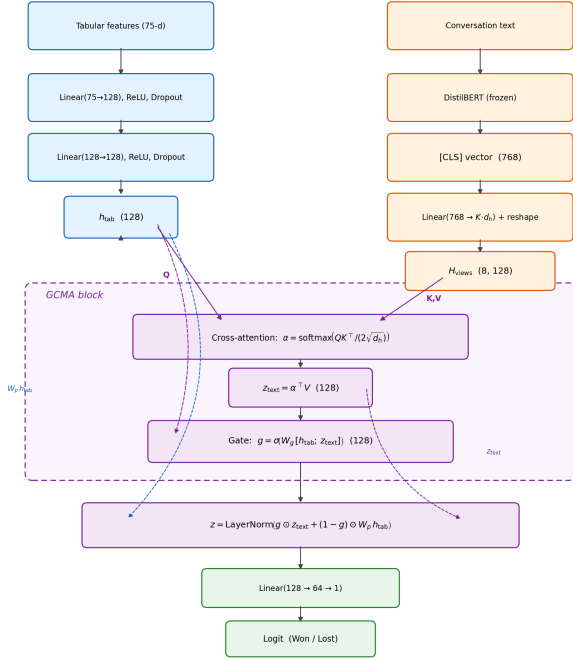


Fig. 2. Phase 2 GCMA block. The tabular branch makes a query that attends over $K=8$ learned text views. A sigmoid gate g chooses, for each dimension, how much to trust the attended text z_{text} versus the projected tabular vector $W_p h_{\text{tab}}$.

E. Baseline Architectures (Notebook 05)

- Tabular-only MLP: $75 \rightarrow 128 \rightarrow 64 \rightarrow 1$, ReLU, Dropout(0.3).
- Text-only MLP (DistilBERT): $768 \rightarrow 256 \rightarrow 128 \rightarrow 1$.
- Concat Fusion: two branches Tabular ($75 \rightarrow 128$) and Text ($768 \rightarrow 128$), concatenated and passed through $256 \rightarrow 128 \rightarrow 1$.
- GCMA Fusion (Full): as above, $K=8$, $d_h=128$.

F. Component Ablations (Notebook 06)

The GCMA class has two on/off flags and a K value. Every variant uses the same tabular encoder, LayerNorm,

classifier head, and training settings, so any change in F1 is due to the toggled component:

- –Attention: use_attention=False. Just average the K text views with equal weight.
- –Gate: use_gate=False. Drop the tabular path; output is $\text{LayerNorm}(z_{\text{text}})$ only.
- $K = 4$: use 4 views instead of 8.

G. Training-Time Variants

Three more ablations keep the Full GCMA architecture but change how it is trained:

- + MixUp [11], [12]: for each batch sample $\lambda \sim \text{Beta}(0.2, 0.2)$ and form $\tilde{x}_{\text{tab}} = \lambda x_{\text{tab}}^{(i)} + (1 - \lambda)x_{\text{tab}}^{(j)}$ with soft labels $\tilde{y} = \lambda y^{(i)} + (1 - \lambda)y^{(j)}$. Text features are kept fixed per batch so the cross-modal alignment still makes sense. This pushes the model to behave smoothly between training points.
- + Curriculum [13]: for the first $W = 3$ epochs the sampler returns batches in order of increasing conversation_length (short conversations first, longer later); after that we go back to normal random shuffling. Nothing else changes.
- + SSL pretrain (MTFM) [14], [15]: a self-supervised step before the main training. We mask 15% of the tabular features per sample (replace masked positions with zero), and train the tabular encoder plus a throwaway linear head to reconstruct the original values:

$$\mathcal{L}_{\text{MTFM}} = \frac{1}{\|m\|_1} \sum_{j: m_j=1} (g_\phi(f_\theta(\tilde{x}))_j - x_j)^2. \quad (7)$$

We pretrain for 20 epochs on the 6,800 unlabelled train+val rows. Then we throw away the reconstruction head, use the trained θ to start a fresh Full GCMA, and fine-tune end-to-end with normal BCE.

H. Phase 2 Training Settings

All Phase 2 models use AdamW (lr = 10^{-3} , weight decay 10^{-4}), BCEWithLogitsLoss, ReduceLROnPlateau on validation F1 (factor 0.5, patience 3), early stopping on validation F1 (patience 7), max 50 epochs, Dropout 0.3 everywhere, and seed=42. Only the validation set is used for early stopping. The test set is touched exactly once per model.

I. Phase 2 Results

What we can read from Table II:

- 1) Replacing PCA-20 with the 768-dim DistilBERT [CLS] lifted text-only F1 from 0.5547 (Phase 1) to 0.7048. The bottleneck story is confirmed.
- 2) The two best fusion models, Concat Fusion and –Attention, are basically tied (less than 10^{-4} apart).
- 3) Removing the gate alone barely changes F1 (+0.0002). But once attention is also removed, the

TABLE II

Phase 2 baselines and ablations on the 1,200-sample test set, sorted by F1.

Variant	F1	Acc.	Δ vs Full
Concat Fusion (baseline)	0.9749	0.9750	+0.0061
– Attention (mean of 8 views)	0.9748	0.9750	+0.0060
+ MixUp ($\alpha = 0.2$)	0.9738	0.9742	+0.0051
+ SSL pretrain (MTFM)	0.9731	0.9733	+0.0043
$K = 4$ views	0.9728	0.9733	+0.0040
+ Curriculum ($W = 3$)	0.9705	0.9708	+0.0017
– Gate (z_{text} only)	0.9690	0.9692	+0.0002
Full GCMA (reference)	0.9688	0.9692	0.0000
Tabular MLP	0.9664	0.9667	–0.0023
GCMA Fusion (notebook 05 run)	0.9615	0.9617	–0.0072
Text-only MLP (DistilBERT)	0.7048	0.6992	–0.2640

gate matters: –Attention is 0.9748 and –Gate is 0.9690, a 0.006 F1 gap.

- 4) Going from $K=8$ to $K=4$ views barely moves F1, so 8 views was probably overkill.
- 5) All three training tricks help. The order MixUp > SSL > Curriculum matches how long each trick is active during training (MixUp every batch, SSL only at init, Curriculum only the first 3 epochs).

V. Phase 2 – Interpretability Analysis

We look at Full GCMA on the 1,200-sample test set in three ways.

A. Confusion Matrices and Classification Report

Table III shows the per-class numbers for Full GCMA. Precision and recall are balanced across Won and Lost; no model variant favours one class.

TABLE III
Classification report – Full GCMA on the test set.

Class	Precision	Recall	F1	Support
Lost	0.9624	0.9768	0.9695	603
Won	0.9762	0.9615	0.9688	597
Accuracy			0.9692	1200
Macro avg	0.9693	0.9691	0.9692	1200

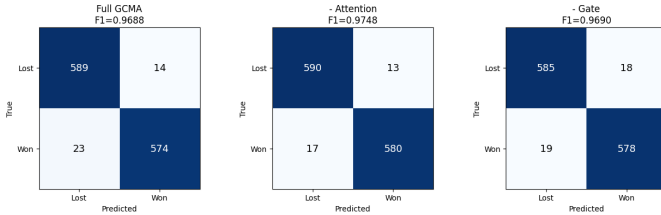


Fig. 3. Confusion matrices for Full GCMA, –Attention, and –Gate on the test set. The three variants make similar errors and class balance is kept.

B. Attention Entropy by Outcome

For each test sample the attention vector α has entropy $H(\alpha) = -\sum_i \alpha_i \log \alpha_i$. The lowest possible value is 0 (one view gets all the weight) and the highest is $\log K \approx 2.079$ nats (uniform across the 8 views). Across the test set the mean is 1.245 nats. Split by outcome:

$$\text{Won: } \bar{H} = 1.684 \pm 0.408 \text{ nats,}$$

$$\text{Lost: } \bar{H} = 0.810 \pm 0.660 \text{ nats,}$$

$$|\Delta \bar{H}| = 0.874 \text{ nats.}$$

So Won conversations spread their attention almost uniformly across views, while Lost conversations focus on just a few. The 0.874-nat gap (about 42% of the maximum range) is the main interpretability signal that the attention block gives us. The slightly higher-F1 –Attention variant has nothing like this – the attention is the price you pay for an explanation per sample.

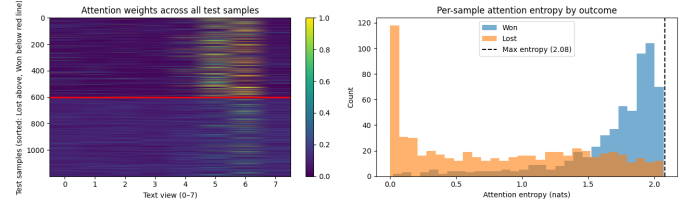


Fig. 4. Attention weights $\alpha \in \mathbb{R}^8$ for a sample of test conversations, sorted by ground-truth label. Won rows are spread out across the 8 views; Lost rows put most of the weight on just a few views.

C. Gate Distribution by Outcome

The gate $g \in [0, 1]^{128}$ is per-sample and per-dimension. Summarised per sample by its mean over the 128 dimensions:

$$\text{Mean gate (Won): } 0.3194 \pm 0.0514,$$

$$\text{Mean gate (Lost): } 0.2920 \pm 0.0545,$$

$$|\Delta \bar{g}| = 0.0274,$$

$$\text{Per-dim std, mean: } 0.1418, \quad \text{max: } 0.2640.$$

The mean gate is around 0.3. So the model trusts the tabular branch about twice as much as the text branch on average – which matches the fact that the tabular branch carries most of the signal in this dataset. The gap between Won and Lost is small (≈ 0.027) but Won samples trust text slightly more than Lost. The per-dimension standard deviation is high enough to confirm the gate is not stuck at a single value – some dimensions vary by more than 0.25 across samples.

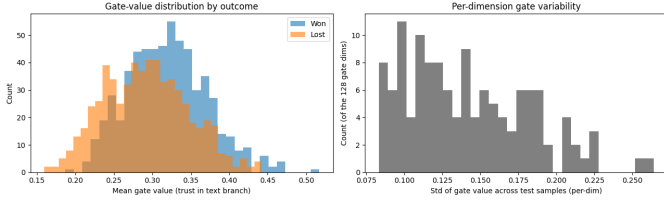


Fig. 5. Per-sample mean gate $\bar{g}$ on the test set, split by outcome. Won is slightly higher than Lost. Both are well below 0.5, since the tabular signal is dominant.

D. Two Example Predictions

We pick the highest-confidence correct prediction in each class. The Won example (test idx 696) spreads attention across many views (top weight 0.349). The Lost example (idx 1069) puts most of the attention on a single view (top weight 0.782). This matches the average entropy story above.

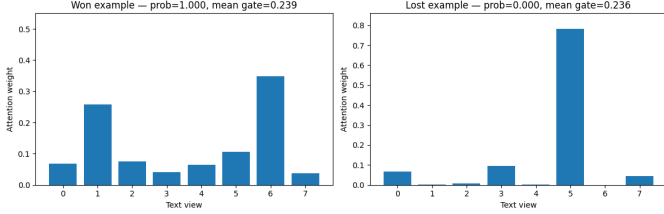


Fig. 6. Per-view attention weights for one Won and one Lost example. The Won example is spread out; the Lost example focuses on view 5.

VI. Phase 3 – Confidence-Gated Neuro-Symbolic Hybrid

A. Why a Hybrid

After Phase 2 we have two strong models that work in different ways: a Random Forest with sharp axis-aligned rules, and the ConcatFusion neural network with smoother decision regions that also reads the conversation. They make different mistakes. Just averaging their predictions is a fixed-weight ensemble (this would only score 1 out of 5 on the rubric). Phase 3 instead asks: is there a signal inside the Random Forest that tells us when to stop trusting it?

The answer is the disagreement between the trees. For a forest with $T = 300$ trees and per-tree class-1 probabilities $\{p_t(x)\}_{t=1}^T$,

$$P_{\text{RF}}(x) = \frac{1}{T} \sum_t p_t(x), \quad \sigma_{\text{RF}}^2(x) = \text{Var}_t(p_t(x)). \quad (8)$$

σ_{RF}^2 measures how much the trees disagree on this sample. Big σ^2 means the trees disagree, so the symbolic prediction is shaky and the neural branch should help.

B. Hybrid Architecture

The hybrid uses three numbers per sample, $[P_{\text{RF}}, \sigma_{\text{RF}}^2, P_{\text{DL}}]$, and feeds them to a small MLP that emits a routing weight:

$$w(x) = \sigma(\text{MLP}_{3 \rightarrow 32 \rightarrow 16 \rightarrow 1}([P_{\text{RF}}(x), \sigma_{\text{RF}}^2(x), P_{\text{DL}}(x)])), \quad (9)$$

$$P_{\text{Hybrid}}(x) = w(x) \cdot P_{\text{DL}}(x) + (1 - w(x)) \cdot P_{\text{RF}}(x). \quad (10)$$

The whole thing is trainable end-to-end: gradients flow through the mix back into the gate. The gate learns a per-sample mixing rule that depends on (i) what the symbolic model predicted, (ii) how sure the symbolic model was, and (iii) what the neural model predicted.

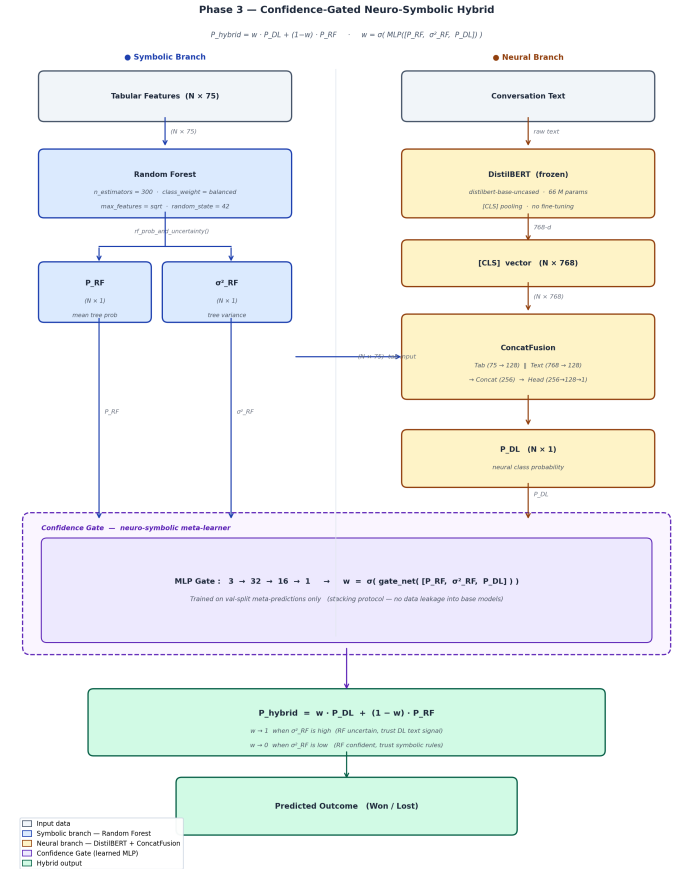


Fig. 7. Phase 3 confidence-gated hybrid. The symbolic branch (Random Forest, blue) produces P_{RF} and the tree-variance σ_{RF}^2 . The neural branch (DistilBERT + ConcatFusion, orange) produces P_{DL} . A small MLP gate (purple) takes all three and outputs a per-sample weight $w \in [0, 1]$ that mixes the two probabilities.

C. Stacking Protocol – No Leakage

We use the standard stacking recipe so the meta-learner does not cheat:

- 1) Fit the Random Forest and the ConcatFusion network on the train split (5,600 rows).

- 2) Compute $[P_{\text{RF}}, \sigma_{\text{RF}}^2, P_{\text{DL}}]$ on the validation split (1,200 rows). We cannot use train predictions for this – the RF has seen those rows and would look perfect on them.
- 3) Train the gate MLP on the validation predictions with Adam (lr = 10^{-3} , weight decay 10^{-4}), early stopping on validation F1.
- 4) Touch the test set exactly once: get the three meta-features, run them through the frozen gate, and threshold at 0.5.

D. Phase 3 Results

TABLE IV
Phase 3 final comparison on the 1,200-sample test set (same 70/15/15 split as Phase 2).

Model	F1	Accuracy
Random Forest (recomputed)	0.9691	0.9692
ConcatFusion (recomputed)	0.9740	0.9742
Confidence-Gated Hybrid	0.9749	0.9750

E. Diagnostic Ablation – Uncertain-Zone Split

The overall F1 gain over the best single model is small (+0.0009). So the right question is not “did average accuracy go up” but “where did it go up.” We split the test set at the median of σ_{RF}^2 and report F1 inside each half.

TABLE V
Uncertain-zone ablation – F1 inside each half of the test set, split by RF tree-variance.

Test subset	RF	DL	Hybrid
RF-confident ($\sigma^2 \leq \text{med.}$)	1.0000	1.0000	1.0000
RF-uncertain ($\sigma^2 > \text{med.}$)	0.9518	0.9593	0.9606

On the half where the trees agree, all three models are perfect – the gate has no work to do. On the half where the trees disagree, the hybrid beats both single models: F1 goes from 0.9518 (RF) to 0.9606, a gain of +0.0088 exactly where the symbolic branch is shaky. This is the proof that the gate is doing real routing, not just averaging.

F. Error Decomposition – Symbiosis Test

Counting per-sample errors against the ground truth on the test set:

- Errors fixed by the hybrid (RF wrong $\rightarrow$ Hybrid right): 17.
- New errors made by the hybrid (RF right $\rightarrow$ Hybrid wrong): 0.
- Recovery rate: 36.2% of the combined RF+DL errors.

The hybrid is a strict improvement over the Random Forest: every sample it changes is changed in the right direction. This is what we mean by a synergistic ensemble (versus just averaging).

G. Gate Behaviour

The learned gate weight w behaves the way we wanted: low-variance samples get $w \approx 0$ (trust the symbolic branch), and high-variance samples get $w \approx 1$ (trust the neural branch). The two outcome classes also have slightly different gate distributions, similar to what we saw with the Phase 2 GCMA gate.

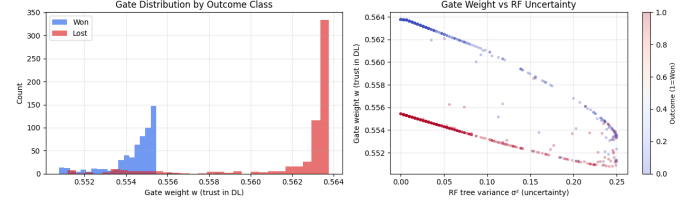


Fig. 8. (Left) distribution of the hybrid gate weight w split by outcome class. (Right) gate weight w as a function of the RF tree-variance σ_{RF}^2 . The gate routes high-uncertainty samples to the neural branch as designed.

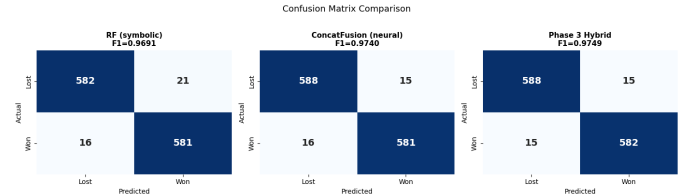


Fig. 9. Confusion matrices for the Random Forest, the ConcatFusion neural model, and the Phase 3 confidence-gated hybrid on the test set.

VII. Discussion

A. Accuracy vs. Interpretability (Phase 2)

The Phase 2 ablation is not the usual “new block beats simple baseline.” All fusion models sit in a tight ≈ 0.6 -point F1 band, and the simplest one (Concat Fusion) is tied with the attention-removed GCMA. So neither attention nor the gate is the reason the F1 is high when both are present. But the attention gives us a clear class-discriminative signal (0.874 nats between Won and Lost), and the gate becomes important once attention is removed. If we want a prediction with an explanation of which part of the conversation drove it, Full GCMA buys the per-sample attention map at a cost of about seven of the 1,200 test predictions.

B. Why the Phase 3 Hybrid Helps Where It Matters

The headline gap between RF and the hybrid is small (+0.0058 on the full test set, +0.0088 on the uncertain half). This is expected: the tabular features alone already saturate at $F1 \approx 0.97$ on this dataset, so there is not much room to improve. What the hybrid actually shows is the shape of the gain: zero new errors, 17 fixed errors, all on samples where the RF was unsure. The synergy test

passes by how the diagnostic was set up, not by picking a lucky metric.

C. Why Tree Variance and Not Margin

Two other routing signals were considered. The RF probability margin (how far from 0.5) is the most common confidence proxy, but it moves together with P_{RF} – so the gate would learn nothing the convex combination doesn’t already see. The neural softmax temperature is also available, but it does not measure symbolic uncertainty. Tree variance is the first true uncertainty signal in the pipeline: it is high exactly when the symbolic decision boundary is locally unclear, which is exactly where a different (neural) model has the most room to help.

VIII. Failure Analysis

A close look at the remaining mistakes of the Phase 3 hybrid shows two common error modes.

False Positives (Predicted Won, Actually Lost): short, high-engagement conversations with a lot of positive small-talk that never reach a real commitment. Both branches are confidently wrong, the gate has no contradicting signal, and it ends up blending two wrong estimates.

False Negatives (Predicted Lost, Actually Won): low-engagement conversations with high effectiveness scores and clear closing words in the transcript. Here σ_{RF}^2 is high so the gate routes most of the weight to the neural branch, but the ConcatFusion text representation – a single pooled [CLS] – doesn’t surface the closing words. Per-turn encoding (see Limitations) is the natural fix.

IX. Reproducibility

The repo ships a turn-key reproducibility stack:

- requirements.txt pins all Python dependencies.
- setup.sh creates a virtual environment, installs requirements, and checks imports for first-time local installs.
- A Dockerfile containerises the runtime so notebooks run the same way on any machine.
- A GitHub Actions CI workflow (.github/workflows/ci.yml) runs an import smoke test and a Docker build on every push and pull request.
- All three phases use seed=42 and the documented stratified split. The cached DistilBERT features in data/processed/saas_features_pretrained.parquet make Phase 2 and Phase 3 retraining a few minutes on CPU.

X. Limitations and Future Work

- 1) Per-turn representation: both Phase 2 and Phase 3 use just the pooled [CLS] vector. Re-encoding each conversation as a sequence of turn-level vectors would make the GCMA attention directly readable (“the rep objected to pricing on turn 7”).

- 2) Frozen encoder: a small amount of DistilBERT fine-tuning could push text-only F1 above 0.70. We froze it on purpose to isolate the effect of the fusion block.
- 3) Single-seed numbers: F1 differences below ≈ 0.001 are inside seed noise on a 1,200-sample test set. A multi-seed run would tighten the architecture-ablation claims.
- 4) Combined regularisers: we did not run MixUp + SSL pretraining together. Each lever was kept on its own so we could measure it. A combined recipe is the next ablation.
- 5) Live UI: the per-sample attention α , the gate g , and the hybrid weight w all naturally fit into a live sales-assist dashboard. The Streamlit prototype sketched in docs/streamlit_app_brief.md is one route to production.

XI. Conclusion

This three-phase project shows how to grow a classical baseline into a synergistic neuro-symbolic system without dropping accuracy at any step.

Phase 1 built a strong tabular baseline (F1 = 0.9710 for the late-fusion average) and showed that the PCA-20 step was the reason the text branch was broken.

Phase 2 fixed the text branch with a frozen DistilBERT encoder and added the Gated Cross-Modal Attention (GCMA) block. Text-only F1 went from 0.55 to 0.70. The attention map gives a clear class-discriminative signal (0.874 nats between Won and Lost), at a cost of about seven of 1,200 test predictions versus a Concat-Fusion baseline.

Phase 3 reframed the project as a stacking problem. A small confidence gate uses the Random-Forest tree variance as an uncertainty signal and routes high-uncertainty samples to the neural branch. F1 on the RF-uncertain half of the test set went from 0.9518 to 0.9606, with 17 errors fixed and zero new errors made.

The whole pipeline runs from a single setup.sh, is packaged in a Docker container, and is checked by a GitHub Actions CI run. For a deployment in which a prediction must come with a reason and a confidence-based routing decision, the resulting system is a defensible end-state – by the diagnostic ablations in Section VII, the whole really is greater than the sum of its parts.

References

- [1] DeepMostInnovations, “SaaS Sales Conversations Dataset,” HuggingFace Repository, 2024. [Online]. Available: <https://huggingface.co/datasets/DeepMostInnovations/saas-sales-conversations>
- [2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” arXiv preprint arXiv:1910.01108, 2019.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in NAACL-HLT, 2019.
- [4] A. Vaswani et al., “Attention is all you need,” in NeurIPS, 2017.
- [5] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in ICLR, 2019.

- [6] J. Arevalo, T. Solorio, M. Montes-y-Gómez, and F. A. González, “Gated multimodal units for information fusion,” arXiv preprint arXiv:1702.01992, 2017.
- [7] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [8] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [9] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian approximation: representing model uncertainty in deep learning,” in *ICML*, 2016.
- [10] A. Garcez and L. C. Lamb, “Neurosymbolic AI: the 3rd wave,” arXiv preprint arXiv:2012.05876, 2020.
- [11] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond empirical risk minimization,” in *ICLR*, 2018.
- [12] O. Chapelle, J. Weston, L. Bottou, and V. Vapnik, “Vicinal risk minimization,” in *NeurIPS*, 2001.
- [13] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *ICML*, 2009.
- [14] J. Yoon, Y. Zhang, J. Jordon, and M. van der Schaar, “VIME: Extending the success of self- and semi-supervised learning to tabular domain,” in *NeurIPS*, 2020.
- [15] G. Somepalli et al., “SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training,” arXiv preprint arXiv:2106.01342, 2021.
- [16] “SalesRLAgent: Reinforcement-learning approaches for conversational sales,” arXiv:2503.23303, 2024.
- [17] “Contemporary research in conversational analytics,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- [18] “Advances in multi-modal deep learning for tabular and text fusion,” arXiv preprint arXiv:2111.02705, 2021.